

Conflicts, Voiding, Finality & Checkpoints

How Hathor resolves two transactions that spend the same coin, undoes a losing branch, and decides when history is settled enough to trust.

SUBJECT hathor-core · Part I · Track B (domain concepts)

CHAPTER 10 · Foundations · Concepts

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Conflict · Double-spend · Voiding · voided_by · Propagation · Reorg · Probabilistic finality · Confirmations · Checkpoints

Table of Contents

Chapter 10 — Conflicts, Voiding, Finality & Checkpoints	3
10.1 Two kinds of "bad" transaction	3
10.2 Voiding: marking, not deleting	4
10.3 Deciding a conflict, and propagation	5
10.4 Reorgs in a DAG	6
10.5 Finality: probabilistic, then anchored	6
10.6 Bridge — voiding and consensus in code	7
Recap	8

Chapter 10 — Conflicts, Voiding, Finality & Checkpoints

What you'll learn in this chapter

- What a **conflict** is in Hathor — concretely, two transactions spending the same output — and how it differs from an invalid transaction.
- **Voiding**: how the node marks a transaction as "not counted" without deleting it, via the `voided_by` metadata field, and why marking beats deleting.
- How a conflict is decided — the heavier transaction wins — and how voiding **propagates** to everything built on the loser.
- **Reorgs** in a DAG: how a heavier competing block-chain takes over, and what becomes of the displaced history.
- Why finality is **probabilistic**, how confirmations harden it, and how **checkpoints** put a hard floor under it.
- A **bridge** to the consensus code that implements all of this.

This chapter closes Track B by making the abstract promise of Chapters 6–9 operational. We have a ledger shaped as a DAG (Ch 8), money tracked as UTXOs (Ch 7), and work measured as weight with "heaviest wins" as the rule (Ch 9). The missing piece is the mechanism: when two transactions collide, or when a competing branch overtakes the leader, what actually happens in the node? The answer is **voiding** — a single idea that handles double-spends, reorgs, and invalidity uniformly. Once you have it, the consensus chapter (Ch 32) is a detailed reading of machinery you already understand.

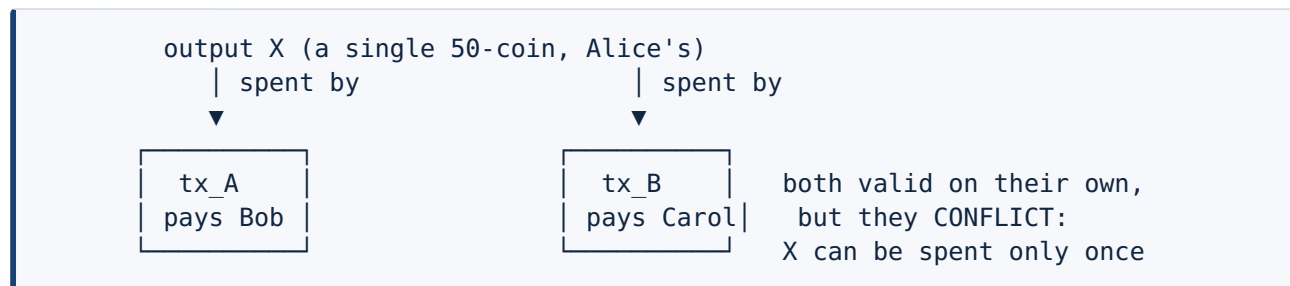
Track B cites Hathor code; this chapter points at the consensus package at the moments the concept becomes a concrete function.

10.1 Two kinds of "bad" transaction

It helps to separate two things newcomers blur together.

An **invalid** transaction breaks a rule on its own — a bad signature, weight below the minimum, malformed structure, inputs that don't cover outputs. Invalidity is detectable by looking at the transaction in isolation, and the node rejects such a transaction at verification (Chapter 31). It never enters the ledger.

A **conflict**¹ is different and subtler: each transaction is individually valid, but they cannot both be true. The canonical case is the **double-spend** (Chapters 6–7): two transactions, each well-formed and correctly signed, that name the same output as an input. Each is fine alone; together they'd spend one coin twice.



Crucially — and this is the DAG difference (Chapter 8 §8.4) — both conflicting transactions can sit in the graph at once, attached via their parent edges. The structure does not forbid the collision; it holds both and waits for consensus to pick a winner. That picking is voiding.

10.2 Voiding: marking, not deleting

When the node detects a conflict (or, as we'll see, when something a transaction depends on goes bad), it does not erase the loser. It **voids**² it: marks it as "present in the graph but not counted as part of the real ledger." A voided transaction's outputs are not spendable, and its effect on balances is undone — but the transaction object stays in storage.

The mark is a single metadata field, `voided_by` (`hathor/transaction/transaction_metadata.py:45`):

```
voided_by: Optional[set[bytes]] # set of hashes causing this vertex to be voided
```

The rule is simple: **if `voided_by` is empty (or `None`), the transaction is executed (counts); if it holds any hashes, the transaction is voided (doesn't count).** The hashes inside say why — which transactions are responsible for the voiding (a conflicting winner, or a voided ancestor). A transaction that loses a conflict gets its own hash placed in its `voided_by` — it is, in a sense, voiding itself by losing.

Why mark instead of delete? Three reasons, each consequential:

1. **Reversibility.** Consensus can change. A transaction voided today might be un-voided tomorrow if the branch it's on becomes the heaviest (a reorg, §10.4). If you'd deleted it, you couldn't bring it back; a flag, you just clear. Marking makes consensus decisions undoable, which a DAG with shifting weights absolutely needs.

2. **Auditability.** The full graph — including the losing sides of past conflicts — remains inspectable. The `voided_by` set records the reason, so the history of why the ledger looks as it does is preserved.
3. **Referential integrity.** Other vertices may point at a now-voided one through parent edges. Deleting it would tear holes in the graph; marking leaves the structure intact and lets voiding propagate cleanly (§10.3).

This single flag — present-but-not-counted, with a reason and a reverse switch — is the mechanism behind double-spend resolution, reorgs, and dependency-driven invalidation alike. That uniformity is the elegance.

10.3 Deciding a conflict, and propagation

When two transactions conflict, which one wins? The heaviest-DAG rule of Chapter 9 decides it. The consensus code's `check_conflicts` (`hathor/consensus/transaction_consensus.py:302`) compares the conflicting transactions by **accumulated weight** — the total work standing behind each (Chapter 9 §9.4):

- The transaction with the **greater accumulated weight wins**: its hash is removed from its `voided_by`, so it executes.
- The loser keeps (or gains) its hash in `voided_by`, so it stays voided.
- Ties — equal accumulated weight — are broken deterministically (by timestamp, so every node reaches the same decision; consensus must be reproducible).

The winner is "the transaction the rest of the network put more work behind," which is exactly the security property we want: to make your double-spend win, you'd have to out-work everyone confirming the honest transaction.

Voiding propagates. A transaction does not stand alone — others spend its outputs and confirm it. So when a transaction is voided, everything that depends on it must be voided too, or the ledger would be inconsistent (you can't have a valid transaction spending a coin from a voided one). The consensus code walks the descendants — both those that spend the voided transaction's outputs and those that confirm it — and propagates the voiding outward (`transaction_consensus.py`, the voiding-propagation walk around `:374`). The walk uses the DAG traversal machinery (Chapter 8 §8.1's topological order) to reach every affected vertex.

```
tx_A voided → every tx spending tx_A's outputs → and THEIR dependents...  
              (voiding cascades down the DAG until it reaches vertices  
              that don't depend on anything voided)
```

This cascade is why marking-not-deleting matters: propagation is just adding hashes to `voided_by` sets down the graph, and if consensus later reverses, the same walk removes them. One mechanism, both directions.

10.4 Reorgs in a DAG

A **reorg**³ (reorganization, Chapter 6 §6.7) is the block-level version of a conflict. Blocks form the ordering backbone (Chapter 8 §8.5), and sometimes a competing chain of blocks — a fork — accumulates more work than the current best chain. When it does, the node must switch: the heavier chain becomes canonical, and the previously-best chain is displaced.

The block-consensus code (`hathor/consensus/block_consensus.py`) handles this by comparing **score** (Chapter 9 §9.4) between competing chain heads; when a new block makes its branch heavier than the current best, that branch wins (with hash used as a deterministic tiebreaker). The switch is recorded as a reorg when the branches diverged before the old best block. Mechanically, the takeover is the same voiding machinery: the blocks and transactions on the losing branch get marked in their `voided_by`, and any transactions that were only on the losing branch become voided (returning their inputs to the unspent set, Chapter 7). If those transactions are still valid and not conflicting, they can be re-applied on the new branch.

One refinement the grounding makes precise: **block voiding does not propagate to child blocks** the way transaction voiding propagates to dependents. A block being voided affects the transactions that depend on it, but the block backbone is handled by the score comparison directly rather than by cascading voids down the block chain (`consensus/consensus.py`). The distinction matters when you read Chapter 32; here, hold the shape: reorg = the heavier block branch wins, the lighter branch's unique transactions get voided.

10.5 Finality: probabilistic, then anchored

Now the question every user actually cares about: **when is my transaction safe?**

As Chapter 6 §6.7 warned, the honest answer is never with absolute certainty, only with growing confidence. A transaction is settled to the degree that work has accumulated behind it (Chapter 9 §9.4). A freshly-attached transaction with little behind it could still lose a conflict or be displaced by a reorg. But as blocks and transactions pile up confirming it, the accumulated weight an attacker would have to overcome grows, and the probability of reversal falls — exponentially with depth, just as in a chain. This is **probabilistic finality**⁴ : confidence asymptotically approaching

certainty, never reaching it. In practice, recipients wait for some number of **confirmations**⁵ — enough confirming work that reversal is economically impossible — before treating a payment as done.

Probabilistic finality has one gap: it says nothing about very deep history being attacked by an adversary willing to spend enormous resources, and it offers no protection while a brand-new node is syncing and hasn't yet seen the work. Hathor closes that gap the way Chapter 6 §6.7 described — with **checkpoints**⁶.

A checkpoint is a hard-coded `(height, hash)` pair — the `Checkpoint` type is literally those two fields (`hathor/checkpoint.py:18`) — declaring "the block at this height is this block, by decree." Checkpoints are shipped in the settings profile (`CHECKPOINTS`, recalled from Chapter 22) and enforced during verification (`verify_checkpoint`, `base_transaction.py`). Their effect:

- **No reorg may rewrite history before a checkpoint.** A competing branch that disagrees with a checkpointed block is rejected outright, however heavy it claims to be. This bounds reorg depth and defeats deep-history attacks.
- **Syncing nodes get trustworthy anchors.** A new node can be sure that the history up to each checkpoint matches everyone else's, without re-deriving all the work from scratch.

The trade-off is honest: checkpoints are a small reintroduction of authority (someone decides what goes in the settings), accepted in exchange for a hard floor under history. They convert the deepest part of the ledger from "probabilistically final" to "final by decree."

10.6 Bridge — voiding and consensus in code

Everything here is implemented in `hathor/consensus/`, toured in Chapter 32. The forward-pointers:

Bridge — conflicts, voiding, finality in the codebase (full treatment in the chapters named):

- **The voided flag.** `voided_by: Optional[set[bytes]]` on `TransactionMetadata` (`transaction_metadata.py:45`) is the present-but-not-counted mark; empty = executed, non-empty = voided. The §1.4 encapsulated-invariant idea applied to consensus state — **Chapters 25 & 32**.
- **Conflict resolution.** `check_conflicts` (`transaction_consensus.py:302`) decides the winner by accumulated weight (Chapter 9), with deterministic tie-breaking — **Chapter 32**.
- **Voiding propagation.** The descendant walk (around `transaction_consensus.py:374`) cascades voiding to dependents and reverses it on un-voiding, using the DAG traversal of Chapter 8 — **Chapters 27 & 32**.
- **Block reorgs.** `block_consensus.py` compares score between chain heads and switches the canonical branch; transaction voiding does the cleanup — **Chapter 32**.
- **The two-tier consensus split.** Hathor separates block consensus (the ordering backbone) from transaction consensus (conflicts among txs) into two modules under `consensus/` ; the top-level `ConsensusAlgorithm` orchestrates them — **Chapter 32**.
- **Checkpoints.** The `Checkpoint(height, hash)` type (`checkpoint.py:18`), sourced from settings (`CHECKPOINTS` , Chapter 22) and enforced by `verify_checkpoint` — **Chapters 22 & 32**.
- **Where ingestion triggers all this.** A newly-accepted vertex runs verification (Chapter 31) and then consensus (this chapter) via the vertex handler — **Chapter 33**.

Recap

Concept	Meaning	In Hathor
Invalid vs. conflict	rule-breaking alone vs. mutually-exclusive	rejected at verify vs. resolved by voiding
Conflict	two valid txs spending one output	the concrete double-spend
Voiding	mark "not counted," don't delete	<code>voided_by</code> set (<code>:45</code>)
Why mark not delete	reversible, auditable, keeps graph intact	un-voiding on reorg

Concept	Meaning	In Hathor
Conflict winner	greater accumulated weight	<code>check_conflicts</code> (:302)
Propagation	voiding cascades to dependents	descendant walk (~:374)
Reorg	heavier block branch takes over	<code>block_consensus.py</code> , by score
Probabilistic finality	confidence grows with confirming work	depth/confirmations
Checkpoint	<code>(height, hash)</code> final-by-decree anchor	<code>checkpoint.py:18</code> , from settings

Voiding is the single mechanism that makes Hathor's consensus work: a present-but-not-counted mark, carrying its own reason and its own reverse switch, that resolves double-spends (the heavier transaction wins, the lighter is voided), cleans up after reorgs (the lighter block branch's unique transactions are voided), and cascades correctly through the DAG because it marks rather than deletes. Finality emerges from accumulated work — probabilistic and depth-hardened — with checkpoints supplying a hard floor where probability alone is not enough. This closes Track B: you now hold the full conceptual ledger — the **problem** (Ch 6), **ownership** as UTXOs (Ch 7), **structure** as a DAG of vertices (Ch 8), **work** measured as weight (Ch 9), and **consensus** as voiding by accumulated weight (Ch 10). Track C turns next to the third foundation — the stack of third-party technologies the node is built from — beginning with the Python platform itself.

1. A conflict is a pair (or set) of individually-valid transactions that cannot all be part of the ledger — classically, two transactions spending the same output (a double-spend). Distinct from an invalid transaction, which breaks a rule on its own. ↵
2. Voiding marks a vertex as not counted toward the real ledger (its outputs unspendable, its effects undone) without deleting it, via the `voided_by` metadata field. Reversible if consensus later changes. ↵
3. A reorg (reorganization) is a switch of the canonical block chain to a competing branch that accumulated more work, displacing the previous branch; transactions unique to the displaced branch are voided. ↵
4. Finality is assurance a transaction cannot be reversed. Hathor's is probabilistic — strengthening with accumulated confirming work — except below a checkpoint, where it is final by decree. ↵
5. Confirmations are the confirming vertices (especially blocks) accumulated on top of a transaction; more confirmations mean more work an attacker must overcome, so recipients wait for a threshold before trusting a payment. ↵
6. A checkpoint is a hard-coded `(height, hash)` anchor declaring a specific block final, so no reorg may rewrite history before it. It bounds reorg depth and gives syncing nodes trustworthy anchors, at the cost of a little reintroduced authority. ↵